

The Hermite ODE and the integer-order parabolic cylinder function: $D_n = e^{-z^2/4}\text{He}_n$ solves Weber

Claude, with Daniel Murfet

2026-09-06

Abstract

Unit 9 of the grammar-paper §4 autoformalisation. We fill two gaps in Mathlib’s Hermite-polynomial API — derivative-lowering $\text{He}'_{n+1} = (n+1)\text{He}_n$ and the Hermite ODE $\text{He}_n'' - X\text{He}_n' + n\text{He}_n = 0$ — then define the integer-order parabolic cylinder function $D_n(z) = e^{-z^2/4}\text{He}_n(z)$ (`eq:Dn_hermite`) and prove it solves the parabolic cylinder equation $f''(z) + (n + \frac{1}{2} - \frac{z^2}{4})f(z) = 0$ (Weber order $\nu = n$), the quantum-harmonic-oscillator content of `lem:qho`. Zero `sorry/axiom`.

1 Setting

Unit 8 proved Weber’s equation for Watanabe’s fluctuation function S_λ (order $\nu = -2\lambda$). This unit is the *integer-order* companion: at $\nu = n \in \mathbb{Z}_{\geq 0}$ the parabolic cylinder function is elementary — a Hermite polynomial times a Gaussian — and this is exactly the QHO wavefunction $\psi_n \propto D_n$ of `lem:qho`. Mathlib has the probabilist Hermite polynomials (`Polynomial.hermite`) with the recurrence $\text{He}_{n+1} = X\text{He}_n - \text{He}_n'$ and the Gaussian–Rodrigues bridge, but not the two derivative identities the ODE needs, so we build them.

2 The things formalised

```
theorem derivative_hermite (n : ℕ) :
  derivative (hermite (n + 1)) = (n + 1) • hermite n

theorem hermite_ode (n : ℕ) :
  derivative (derivative (hermite n)) = X * derivative (hermite n) - n • hermite n

noncomputable def parCylNat (n : ℕ) (z : ℝ) : ℝ :=
  Real.exp (-z ^ 2 / 4) * aeval z (hermite n)

theorem parCylNat_weber (n : ℕ) (z : ℝ) :
  deriv (deriv (fun z => parCylNat n z)) z
  + ((n : ℝ) + 1 / 2 - z ^ 2 / 4) * parCylNat n z = 0
```

The paper writes $D_n(z) = 2^{-n/2}e^{-z^2/4}H_n(z/\sqrt{2})$ with the physicist H_n ; via $H_n(x) = 2^{n/2}\text{He}_n(x/\sqrt{2})$ this is $e^{-z^2/4}\text{He}_n(z)$, the probabilist form Mathlib supports and `parCylNat` uses.

3 Proof strategy

Derivative-lowering. Induction on n . The step differentiates $\text{He}_{n+2} = X\text{He}_{n+1} - \text{He}'_{n+1}$ and, crucially, rewrites the stray He'_n that appears using `hermite_succ` n in the form $\text{He}'_n = X\text{He}_n - \text{He}_{n+1}$ — avoiding any reference to He_{n-1} , so the induction hypothesis $\text{He}'_{n+1} = (n+1)\text{He}_n$ is all that is needed; `ring` closes the polynomial identity.

Hermite ODE. Immediate from derivative-lowering: for $n = m+1$, $\text{He}''_{m+1} = ((m+1)\text{He}_m)' = (m+1)\text{He}'_m = (m+1)(X\text{He}_m - \text{He}_{m+1}) = X\text{He}'_{m+1} - (m+1)\text{He}_{m+1}$; `map_nsmul`, `smul_sub`, `mul_smul_comm` handle the $\bullet$ -bookkeeping.

Weber for D_n . Mirrors the S_λ Weber unit. Transfer the ODE through `aeval` (`rw [hermite_ode]; simp`) to $P_2(z) = zP_1(z) - nP_0(z)$ with $P_j(z) = \text{aeval } z (\text{He}_n^{(j)})$; take two derivatives of $e^{-z^2/4}P_0$ using `Polynomial.hasDerivAt_aeval` for the polynomial factor and the Gaussian derivative, assembling each combinator value with `HasDerivAt.congr_deriv` (by `ring`) and turning the first derivative into a function equality by `funext ∘ HasDerivAt.deriv`; substitute the ODE and `ring` kills the $(n + \frac{1}{2} - \frac{z^2}{4})$ coefficient.

4 Roadblocks and resolutions

Very few, thanks to unit 8’s playbook:

- Derivative-lowering’s induction step naively wants He_{n-1} (not available from the $n+1$ hypothesis); rewriting the offending He'_n via `hermite_succ` n keeps everything in $\{\text{He}_n, \text{He}_{n+1}\}$ so `ring` finishes.
- `Polynomial.hasDerivAt_aeval` takes the polynomial explicitly (dot form `(hermite n).hasDerivAt_aeval z`); passing the point first mis-parses it as the polynomial.
- The same `HasDerivAt.congr_deriv` idiom from unit 8 sidesteps the `smulRight` wrapper and the `Module`-instance diamond; the whole file compiled on the first build.

5 What was Mathlib, what was new

`Mathlib` gave `Polynomial.hermite/hermite_succ`, `Polynomial.hasDerivAt_aeval`, the `exp/product` derivative combinators, and `HasDerivAt.congr_deriv`. New: both Hermite derivative identities (`Mathlib` has the recurrence but neither the lowering rule nor the ODE), the definition of the integer-order D_n against `Mathlib`’s Hermite, and the parabolic cylinder / Weber equation it satisfies — the integer-order companion of the S_λ Weber result.

6 Lessons

When an induction step reaches for the “previous-previous” term, look for a recurrence that re-expresses the derivative in terms of the current and next terms only — here `hermite_succ` turns He'_n into $X\text{He}_n - \text{He}_{n+1}$, collapsing a two-step induction into a one-step one. And the unit 8 differentiation playbook (explicit first-derivative function via `funext ∘ HasDerivAt.deriv`, `congr_deriv` for value normalisation) transfers verbatim to any $e^{-z^2/4} \times (\text{smooth})$ Weber problem.

7 Follow-ups

The remaining §4 items are the operator/algebra ones: the ladder operators $b, b^\dagger$ and the canonical commutator $[b, b^\dagger] = 1$, and the generating function for log insertions (`lem:log_generating`) — these need a Weyl-algebra / operator framework rather than more special-function analysis.